

Repository Package

The `repository` package is responsible for data persistence. It abstracts the underlying storage mechanism from the rest of the application.

Files

`TroubleTicketRepository.java`

An interface defining the contract for Trouble Ticket persistence operations.

```
public interface TroubleTicketRepository {
    List<TroubleTicket> findAll();
    Optional<TroubleTicket> findById(String id);
    TroubleTicket save(TroubleTicket ticket);
    boolean deleteById(String id);
}
```

Explanation

- `findAll()`: Returns all stored tickets.
 - `findById(String id)`: Returns an `Optional` containing the ticket if found, or empty if not.
 - `save(TroubleTicket ticket)`: Persists a ticket (used for both create and update).
 - `deleteById(String id)`: Removes a ticket and returns true if it existed.
-

`InMemoryTroubleTicketRepository.java`

A thread-safe, in-memory implementation of the `TroubleTicketRepository`.

```
public class InMemoryTroubleTicketRepository implements TroubleTicketRepository {

    private final Map<String, TroubleTicket> database = new ConcurrentHashMap<>();

    @Override
    public List<TroubleTicket> findAll() {
        return new ArrayList<>(database.values());
    }

    @Override
    public Optional<TroubleTicket> findById(String id) {
        return Optional.ofNullable(database.get(id));
    }

    @Override
```

```

    public TroubleTicket save(TroubleTicket ticket) {
        database.put(ticket.getId(), ticket);
        return ticket;
    }

    @Override
    public boolean deleteById(String id) {
        return database.remove(id) != null;
    }
}

```

Line-by-line Explanation

- **database = new ConcurrentHashMap<>():** Initializes a thread-safe map to store tickets where the key is the ticket ID.
 - **findAll():** Converts the map values into a new `ArrayList` to avoid exposing the internal map structure and to provide a mutable copy to the caller.
 - **findById(...):** Uses `Optional.ofNullable` to safely handle cases where the ID might not exist in the map.
 - **save(...):** Inserts or updates the ticket in the map using its ID as the key.
 - **deleteById(...):** Removes the entry from the map and uses the return value of `remove` to determine if the ticket was present.
-

Analysis

Data Types

- **ConcurrentHashMap:** Favored over a standard `HashMap` because the API is intended to be used in a multi-threaded web server environment. `ConcurrentHashMap` allows for high concurrency of retrievals and updates without the need for manual synchronization.
- **Optional<TroubleTicket>:** Used for the return type of `findById` to explicitly signal that a result might be absent, forcing the caller (the service layer) to handle the null case safely.
- **ArrayList:** Used in `findAll` to return a snapshot of the data.

Architectural Decisions

- **Repository Pattern:** By using an interface, the application logic is decoupled from the storage technology. This makes it trivial to replace the `InMemory` implementation with a database-backed implementation (e.g., using JPA/Hibernate) in the future.

- **In-Memory Storage:** Ideal for prototyping and development as it requires zero setup (no database installation), though data is lost when the application restarts.